

Presentatie-spiekbrief

Keuzedeel Scouting — webapp: technieken & code-uitleg

React · TypeScript · Vite · React Router · CSS

1. De techniek-stack (waar is het mee gebouwd?)

- **React** JavaScript-bibliotheek die de interface opbouwt uit losse, herbruikbare "componenten".
- **TypeScript** JavaScript met type-controle: fouten worden al tijdens het typen zichtbaar.
- **Vite** De dev-server en "bouwmachine"; toont wijzigingen direct in de browser.
- **React Router** Regelt de navigatie tussen pagina's zonder dat de pagina echt herlaadt.
- **localStorage** Opslag in de browser om ingevulde scores te bewaren na verversen.

Kernbegrip: dit is een Single Page Application (SPA). Er is technisch maar één HTML-pagina; React wisselt de inhoud dynamisch, waardoor het snel aanvoelt als losse pagina's.

2. Componenten, props & de .map()-truc

In plaats van elke pagina apart te schrijven, zijn er herbruikbare componenten gemaakt. De kerngedachte van React: "schrijf één keer, gebruik overal".

De navigatiebalk genereert zichzelf uit een lijst met een lus (.map()). Eén tab toevoegen = één regel aanpassen, in plaats van alle pagina's.

NavBar.tsx — knoppen worden automatisch gegenereerd

```
const pages = [
  { key: 'uitleg', label: 'Uitleg', suffix: '' },
  { key: 'taak-1', label: 'Taak 1', suffix: '/taak-1' },
];

{pages.map(p => (
  <button className={`nav-button ${p.key === active ? 'active' : ''}`}>
    {p.label}
  </button>
))}
```

Nog krachtiger: TaakPagina en ScorePagina zijn één component dat voor beide groepen werkt. De verschillen (titels, taken) worden er als "props" ingestopt — instellingen die je meegeeft, net als argumenten aan een functie.

3. Interactiviteit: state & localStorage

Als iemand een cijfer invult, moet de app dat onthouden. Daarvoor gebruikt React "state" via de useState-hook. Verandert de state, dan tekent React het scherm automatisch opnieuw.

TaakPagina.tsx — scores onthouden in de browser

```
const [scores, setScores] = useState([]); // het geheugen

// useEffect = 'luisteraar': reageert zodra scores veranderen
useEffect(() => {
  localStorage.setItem(storageKey, JSON.stringify(scores));
}, [scores]);
```

Op de scorepagina worden die opgeslagen cijfers weer opgeteld en omgezet in de gekleurde voortgangsbalken (groen / geel / rood).

4. De styling-technieken

- **CSS-variabelen** Kleuren en maten staan één keer bovenaan gedefinieerd (--blue, --yellow, --radius-md). Overal gebruik je var(--blue). Eén plek wijzigen = hele site verandert mee. Dit is een "design-systeem".
- **Flexbox** Moderne layout-techniek (display: flex) om elementen netjes te centreren, verdelen en te laten "wrappen".
- **Media queries** Met @media (max-width: 640px) maak je aparte regels voor kleine schermen (responsive design).
- **Gradients & schaduw** linear-gradient voor kleurverloop, box-shadow voor diepte.
- **Transitions** Vloeiende animaties: bij hover komt een knop zachtjes omhoog.
- **Google Fonts** Het lettertype "Nunito" ingeladen voor een vriendelijke, bij scouting passende uitstraling.

App.css / shared.css — design-tokens + hover-animatie

```
:root {
  --blue: #1A368D;
  --yellow: #FFD549;
  --radius-md: 16px;
}

.next-button { transition: all 0.2s ease; }
.next-button:hover {
  transform: translateY(-3px);      // knop 3px omhoog
  border-color: var(--yellow);
}
```

5. In één zin (de elevator pitch)

React + TypeScript voor de logica, opgebouwd uit herbruikbare componenten met props en state. React Router voor navigatie, localStorage om scores te bewaren. Voor de styling: CSS-variabelen als design-systeem, Flexbox voor de layout, media queries voor responsive gedrag, en gradients / schaduwen / transitions voor een moderne, verzorgde look.

6. Mogelijke vragen van je publiek (met antwoord)

V: Waarom React en niet gewoon losse HTML-pagina's?

A: React laat je componenten hergebruiken en houdt de interface automatisch in sync met de data. Minder herhaling, minder fouten, en de app voelt sneller omdat pagina's niet echt herladen.

V: Wat is het verschil tussen "props" en "state"?

A: Props geef je van buitenaf mee aan een component (vast, zoals instellingen). State is het interne geheugen dat kan veranderen door interactie, bijvoorbeeld de ingevulde scores.

V: Waar worden de scores bewaard? Verdwijnen ze bij verversen?

A: Nee. Ze staan in localStorage — opslag in de browser zelf. Daardoor blijven ze bewaard, ook na het verversen of sluiten van het tabblad.

V: Hoe hebben jullie ervoor gezorgd dat het op mobiel ook werkt?

A: Met "responsive design": Flexbox laat elementen automatisch onder elkaar vallen, en media queries passen maten en lettergroottes aan voor kleine schermen.

V: Hoe blijft de styling consistent en makkelijk aanpasbaar?

A: Door CSS-variabelen (een design-systeem): alle kleuren en maten staan op één plek. Eén kleur wijzigen past meteen de hele site aan.